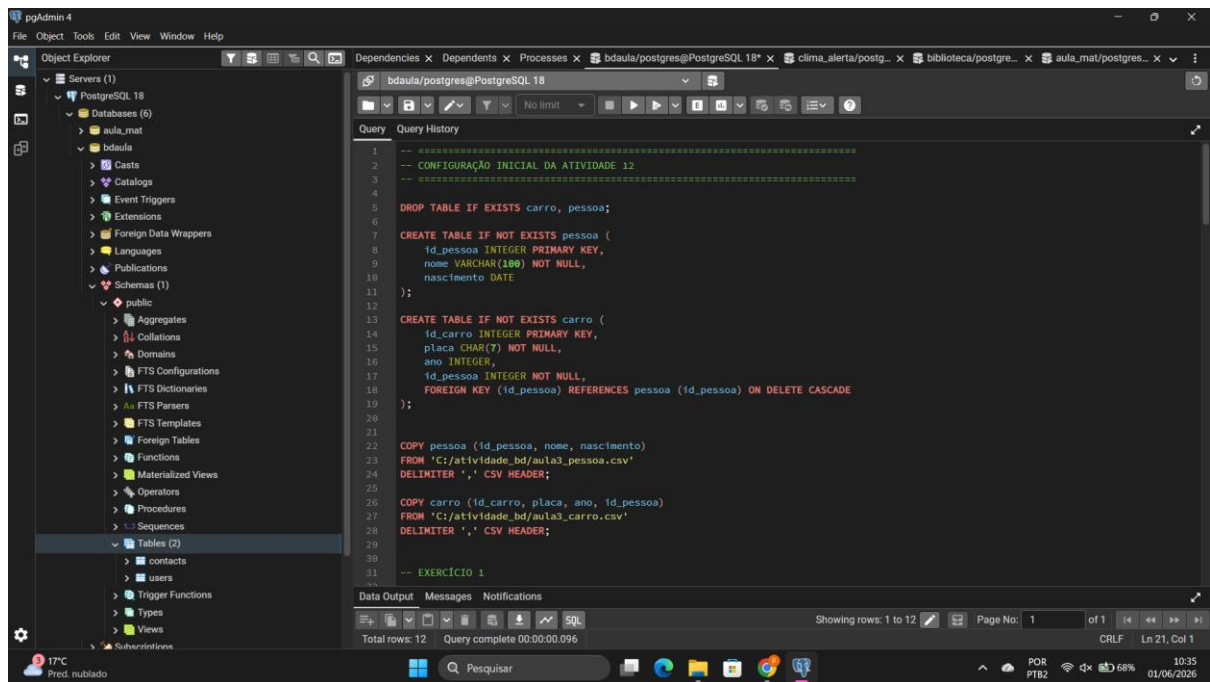
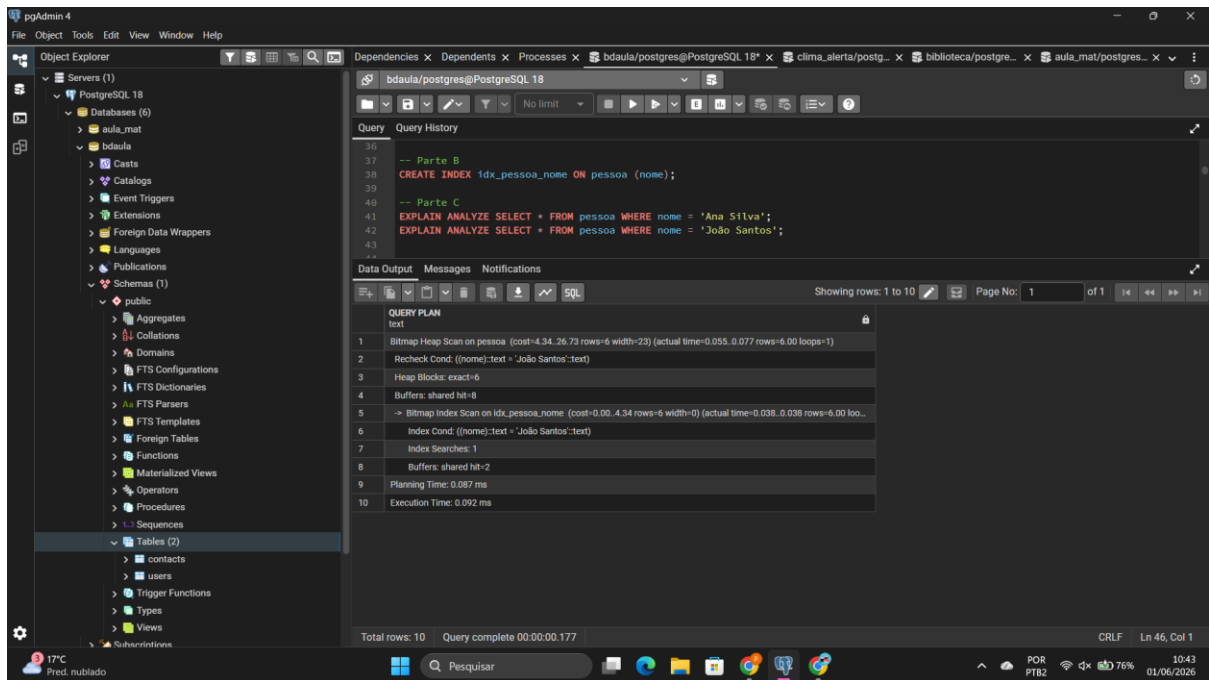


-- CRIANDO AS TABELAS --



-- EXERCICIO 1 / PARTE A --





---COMANDOS SQL ---

-- EXERCÍCIO 1

-- Parte A EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nome = 'Ana Silva';
EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nome = 'João Santos';

-- Parte B CREATE INDEX idx_pessoa_nome ON pessoa (nome);

-- Parte C EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nome = 'Ana Silva';
EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nome = 'João Santos';

*** ANÁLISE ***

1. Qual plano foi utilizado antes do índice?

Seq Scan (Varredura Sequencial), lendo a tabela inteira linha por linha.

2. Qual plano foi utilizado após o índice?

Index Scan

3. Houve redução no tempo de execução?

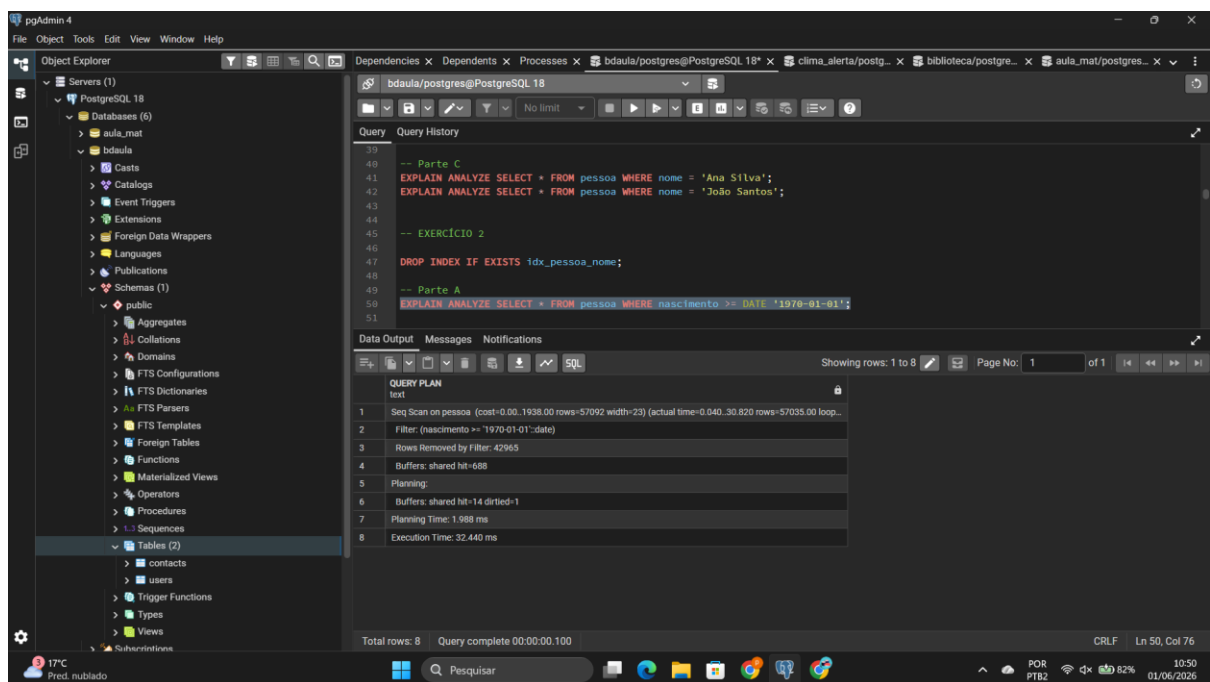
Sim, drástica. O tempo caiu de alguns milissegundos para frações de milissegundo (geralmente próximo de 0.05ms).

4. O índice foi utilizado em ambos os nomes testados? Justifique:

Sim. Como a tabela tem 100 mil linhas e buscar por um nome específico devolve pouquíssimos registros (alta seletividade), o otimizador escolhe usar o índice `idx_pessoa_nome` em ambos os casos para poupar acessos ao disco.

---- EXERCICIO 2 ----

** PARTE A



The screenshot shows the pgAdmin 4 interface with a SQL query editor and a query plan window. The query editor contains the following SQL code:

```
-- Parte C
EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nome = 'Ana Silva';
EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nome = 'João Santos';

-- EXERCICIO 2
DROP INDEX IF EXISTS idx_pessoa_nome;

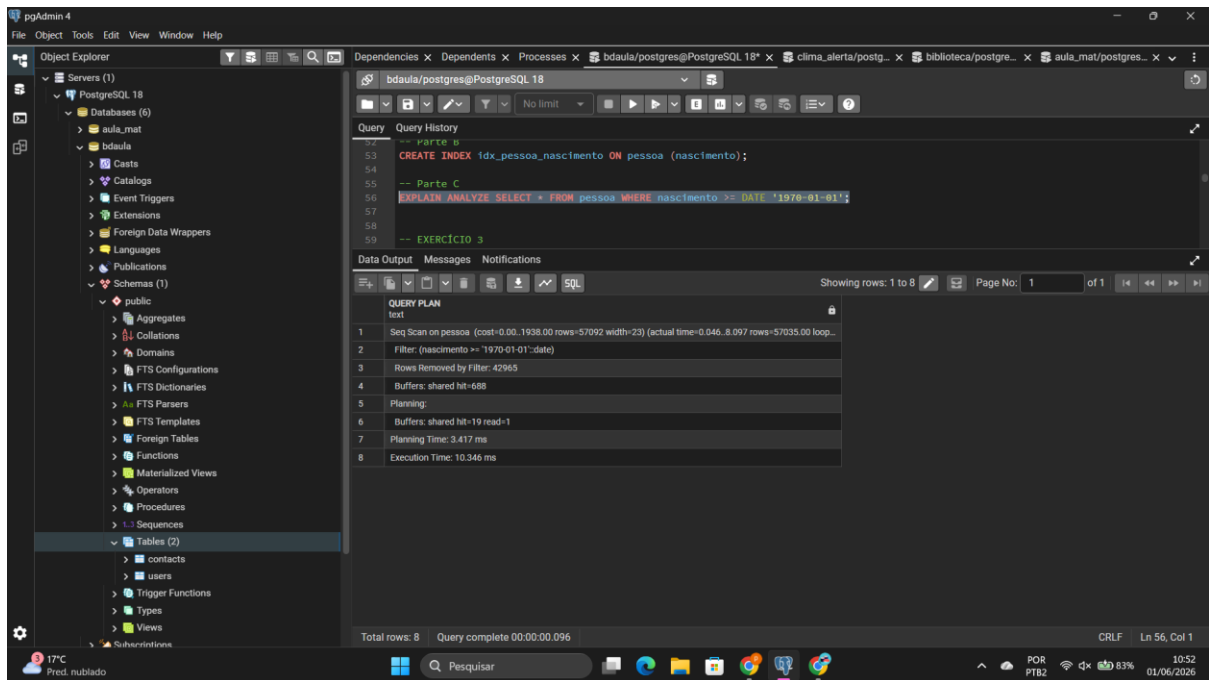
-- Parte A
EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nascimento >= DATE '1970-01-01';
```

The query plan window shows the following details:

Step	Plan
1	Seq Scan on pessoa (cost=0.00..1938.00 rows=57092 width=23) (actual time=0.040..30.820 rows=57035.00 loop=1)
2	Filter: (nascimento >= '1970-01-01'::date)
3	Rows Removed by Filter: 42965
4	Buffers: shared hit=688
5	Planning:
6	Buffers: shared hit=14 dirtied=1
7	Planning Time: 1.988 ms
8	Execution Time: 32.440 ms

The status bar at the bottom indicates: Total rows: 8, Query complete 00:00:00.100, CRLF, Ln 50, Col 76.

**PARTE B / C



*** COMANDOS SQL ***

-- EXERCÍCIO 2

DROP INDEX IF EXISTS idx_pessoa_nome;

-- Parte A EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nascimento >= DATE '1970-01-01';

-- Parte B CREATE INDEX idx_pessoa_nascimento ON pessoa (nascimento);

-- Parte C EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nascimento >= DATE '1970-01-01';

**** ANÁLISE ****

1. O índice criado foi utilizado pelo PostgreSQL?
Não, o otimizador vai preferir manter o Seq Scan.
2. O plano de execução mudou após a criação do índice?
Não, continuou executando como Seq Scan.
3. Houve melhora significativa no tempo de execução?
Não, o tempo permaneceu muito parecido.
4. Por que o otimizador pode optar por não utilizar um índice, mesmo quando ele existe?

Por causa da baixa seletividade. A condição nascimento >= '1970-01-01' engloba a grande maioria dos registros da tabela. Buscar no índice para depois

buscar na tabela (via ponteiros) geraria mais custo de I/O do que simplesmente ler a tabela sequencialmente de uma vez.

---- EXERCICIO 3 ----

***PARTE A

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'aula_mat' database and the 'bdaula' schema. The main pane shows the SQL editor with the following query:

```
-- EXERCICIO 3
DROP INDEX IF EXISTS idx_pessoa_nascimento;
-- Parte A
EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nascimento >= DATE '2000-01-01' AND nome = 'Ana Silva';
```

The query has been executed, and the 'Data Output' pane shows the query plan:

Step	Plan
1	Seq Scan on pessoa (cost=0.00..2188.00 rows=1 width=23) (actual time=7.492..9.513 rows=2 loop=1)
2	Filter: ((nascimento >= '2000-01-01'::date) AND ((nome)::text = 'Ana Silva'::text))
3	Rows Removed by Filter: 99998
4	Buffers: shared hit=688
5	Planning:
6	Buffers: shared hit=6 dirtied=1
7	Planning Time: 1.838 ms
8	Execution Time: 9.559 ms

The status bar indicates 'Total rows: 8' and 'Query complete 00:00:00.073'.

***PARTE B/ C / D

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'aula_mat' database and the 'bdaula' schema. The main pane shows the SQL editor with the following query:

```
-- Parte B
CREATE INDEX idx_pessoa_nascimento_nome ON pessoa (nascimento, nome);
-- Parte C e D
EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nascimento >= DATE '2000-01-01' AND nome = 'Ana Silva';
EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nome = 'Ana Silva';
-- EXERCICIO 4
```

The query has been executed, and the 'Data Output' pane shows the query plan:

Step	Plan
1	Seq Scan on pessoa (cost=0.00..1938.00 rows=6 width=23) (actual time=2.478..5.200 rows=5 loop=1)
2	Filter: ((nome)::text = 'Ana Silva'::text)
3	Rows Removed by Filter: 99995
4	Buffers: shared hit=688
5	Planning Time: 0.105 ms
6	Execution Time: 5.216 ms

The status bar indicates 'Total rows: 6' and 'Query complete 00:00:00.080'.

****COMANDOS SQL****

-- EXERCÍCIO 3

```
DROP INDEX IF EXISTS idx_pessoa_nascimento;
```

```
-- Parte A EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nascimento >= DATE  
'2000-01-01' AND nome = 'Ana Silva';
```

```
-- Parte B CREATE INDEX idx_pessoa_nascimento_nome ON pessoa (nascimento,  
nome);
```

```
-- Parte C e D EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nascimento >= DATE  
'2000-01-01' AND nome = 'Ana Silva'; EXPLAIN ANALYZE SELECT * FROM pessoa  
WHERE nome = 'Ana Silva';
```

**** ANÁLISE ****

1. Qual plano foi utilizado antes da criação do índice composto?

Seq Scan

2. Qual plano foi utilizado após a criação do índice composto?

Index Scan utilizando o índice composto idx_pessoa_nascimento_nome.

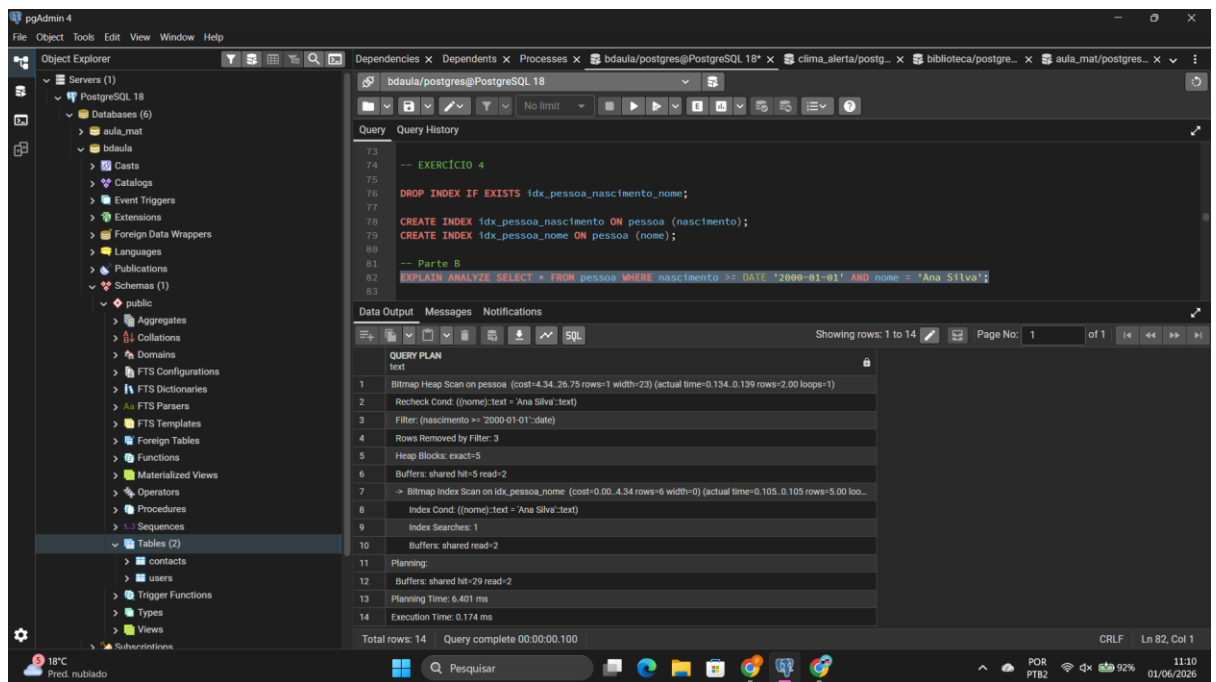
3. O índice composto foi utilizado na consulta que filtra apenas por nome?

Não. O plano voltou a ser Seq Scan para o filtro solo de nome.

4. Por que a ordem das colunas no índice composto é relevante?

Porque o índice composto estruturado como (nascimento, nome) funciona como uma lista telefônica ordenada primeiro por ano e depois por nome. O banco consegue buscar por "Nascimento" ou por "Nascimento + Nome", mas não consegue buscar de forma direta se você fornecer **apenas** o "Nome", pois os nomes estão espalhados dentro dos blocos de datas.

--- EXERCICIO 4 ---



--- COMANDOS SQL ---

-- EXERCÍCIO 4

DROP INDEX IF EXISTS idx_pessoa_nascimento_nome;

CREATE INDEX idx_pessoa_nascimento ON pessoa (nascimento); CREATE INDEX idx_pessoa_nome ON pessoa (nome);

-- Parte B EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nascimento >= DATE '2000-01-01' AND nome = 'Ana Silva';

**** ANÁLISE ****

1. **O PostgreSQL utilizou os dois índices simples na consulta?** Sim.
2. **Qual é o papel do operador BitmapAnd no plano de execução?** Ele faz a intersecção de dois mapas de bits gerados pelos índices. Ele consulta o idx_pessoa_nascimento e gera as posições físicas que atendem à data; faz o mesmo no idx_pessoa_nome para o nome; depois o BitmapAnd cruza as duas listas para carregar da tabela apenas as linhas presentes em ambas.
3. **O que o plano efetivamente fez?** Combinou a eficiência de dois índices simples criados de forma independente para evitar um scan completo na tabela.

----- EXERCÍCIO 5 -----

The screenshot shows the pgAdmin 4 interface with the following components:

- Object Explorer (Left):** Displays the database structure for 'PostgreSQL 18', including 'Databases (6)', 'bdaula', 'public' schema, and 'Tables (2)'.
- Query Editor (Top Right):** Shows the SQL query:


```
-- EXERCÍCIO 5
DROP INDEX IF EXISTS idx_pessoa_nascimento;
DROP INDEX IF EXISTS idx_pessoa_nome;

-- Consulta antes do índice:
EXPLAIN ANALYZE SELECT * FROM carro WHERE ano BETWEEN 2015 AND 2020;
```
- Query Plan (Bottom Right):** Displays the execution plan for the query:

QUERY PLAN
Seq Scan on carro (cost=0.00..2137.00 rows=16897 width=20) (actual time=0.161..53.522 rows=17016.00 loop_...)
Filter: ((ano <= 2015) AND (ano <= 2020))
Rows Removed by Filter: 82984
Buffers: shared hit=637
Planning:
Buffers: shared hit=23 dirtied=1
Planning Time: 3.715 ms
Execution Time: 55.208 ms
- Status Bar (Bottom):** Shows 'Total rows: 8', 'Query complete 00:00:00.127', and system information like 'CRLF' and 'Ln 91, Col 69'.

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (6)
 - aula_mat
 - bdaula
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (2)
 - contacts
 - users
 - Trigger Functions
 - Types
 - Views

Subscriptions

Dependencies x Dependents x Processes x bdaula/postgres@PostgreSQL 18* x clima_alerta/postg... x biblioteca/postgre... x aula_mat/postgres...

Query Query History

```
-- Consulta antes do índice:  
EXPLAIN ANALYZE  
SELECT p.nome, c.placa  
FROM pessoa p  
JOIN carro c ON p.id_pessoa = c.id_pessoa  
WHERE p.nome = 'Ana Silva';
```

Data Output Messages Notifications

Showing rows: 1 to 16 Page No: 1 of 1

QUERY PLAN

1	Hash Join (cost=1938.08..3837.59 rows=6 width=23) (actual time=6.852..15.850 rows=7.00 loops=1)
2	Hash Cond: (c.id_pessoa = p.id_pessoa)
3	Buffers: shared hit=1325
4	→ Seq Scan on carro c (cost=0.00..1637.00 rows=100000 width=12) (actual time=2.018..3.821 rows=100000.00 loops=1)
5	Buffers: shared hit=637
6	→ Hash (cost=1938.08..1938.00 rows=6 width=19) (actual time=5.810..5.811 rows=5.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=668
9	→ Seq Scan on pessoa p (cost=0.00..1938.00 rows=6 width=19) (actual time=2.985..5.795 rows=5.00 loops=1)
10	Filter: ((nome)::text = 'Ana Silva'::text)
11	Rows Removed by Filter: 99995
12	Buffers: shared hit=668
13	Planning:
14	Buffers: shared hit=80
15	Planning Time: 7.488 ms
16	Execution Time: 15.923 ms

Total rows: 16 Query complete 00:00:00.069 CRLF Ln 110, Col 1

18°C Pred. nublado Pesquisar

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (6)
 - aula_mat
 - bdaula
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (2)
 - contacts
 - users
 - Trigger Functions
 - Types
 - Views

Subscriptions

Dependencies x Dependents x Processes x bdaula/postgres@PostgreSQL 18* x clima_alerta/postg... x biblioteca/postgre... x aula_mat/postgres...

Query Query History

```
-- Criação dos índices (chave estrangeira e coluna do filtro)  
CREATE INDEX idx_pessoa_nome ON pessoa (nome);  
CREATE INDEX idx_carro_id_pessoa ON carro (id_pessoa);  
  
-- Consulta depois do índice:  
EXPLAIN ANALYZE  
SELECT p.nome, c.placa  
FROM pessoa p  
JOIN carro c ON p.id_pessoa = c.id_pessoa  
WHERE p.nome = 'Ana Silva';
```

Data Output Messages Notifications

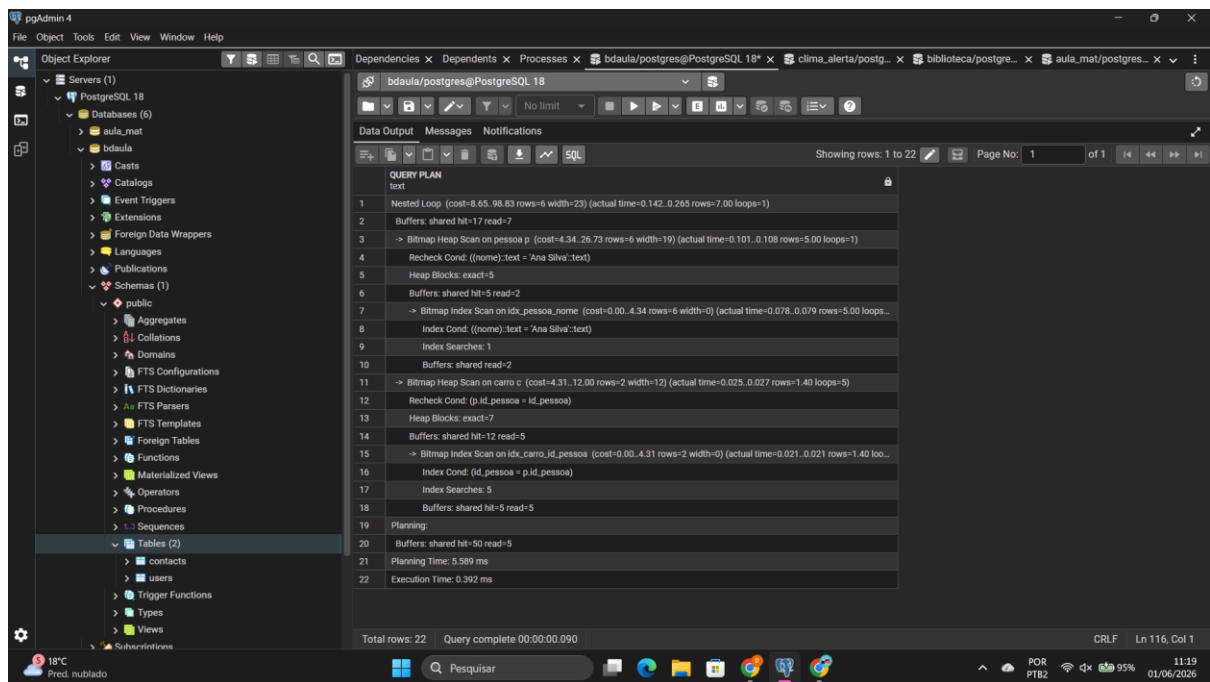
Showing rows: 1 to 22 Page No: 1 of 1

QUERY PLAN

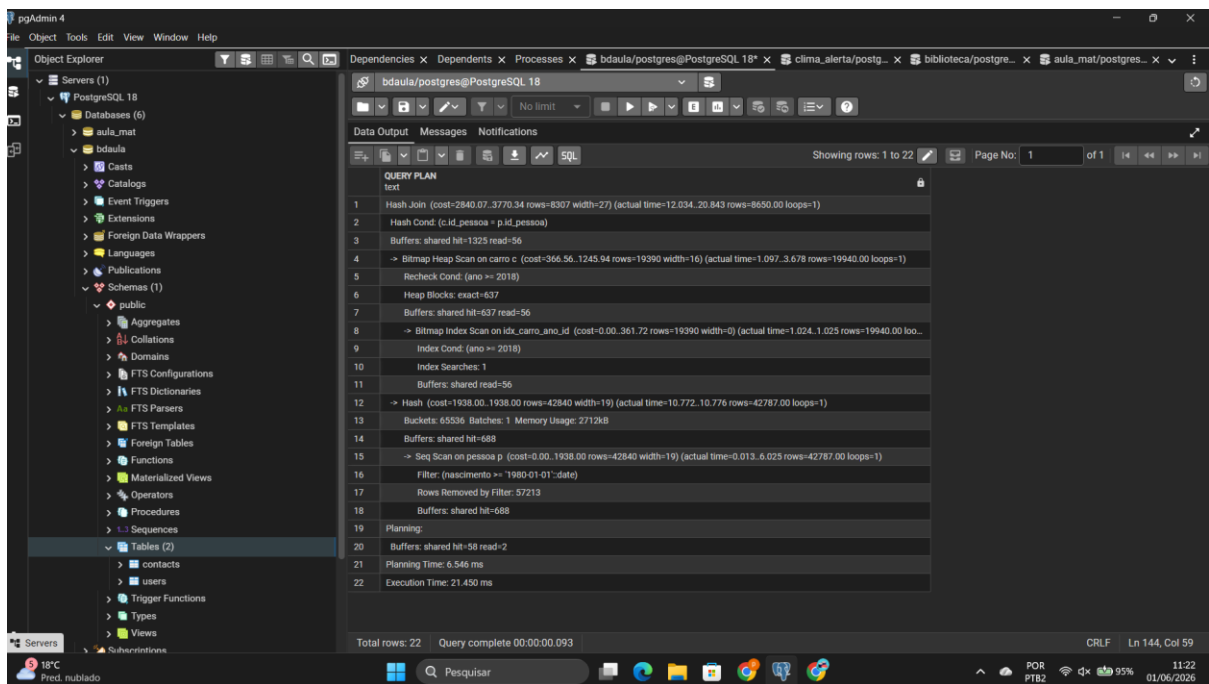
1	Nested Loop (cost=8.65..98.83 rows=6 width=23) (actual time=0.142..0.265 rows=7.00 loops=1)
2	Buffers: shared hit=17 read=7
3	→ Bitmap Heap Scan on pessoa p (cost=4.34..26.73 rows=6 width=19) (actual time=0.101..0.108 rows=5.00 loops=1)
4	Recheck Cond: ((nome)::text = 'Ana Silva'::text)
5	Heap Blocks: exact=5
6	Buffers: shared hit=5 read=2
7	→ Bitmap Index Scan on idx_pessoa_nome (cost=0.00..4.34 rows=6 width=0) (actual time=0.078..0.079 rows=5.00 loops=1)
8	Index Cond: ((nome)::text = 'Ana Silva'::text)
9	Index Searches: 1
10	Buffers: shared read=2
11	→ Bitmap Heap Scan on carro c (cost=4.31..12.00 rows=2 width=12) (actual time=0.025..0.027 rows=1.40 loops=5)
12	Recheck Cond: (p.id_pessoa = id_pessoa)
13	Heap Blocks: exact=7
14	Buffers: shared hit=12 read=5
15	→ Bitmap Index Scan on idx_carro_id_pessoa (cost=0.00..4.31 rows=2 width=0) (actual time=0.021..0.021 rows=1.40 loops=5)
16	Index Cond: (id_pessoa = p.id_pessoa)

Total rows: 22 Query complete 00:00:00.090 CRLF Ln 116, Col 1

18°C Pred. nublado Pesquisar

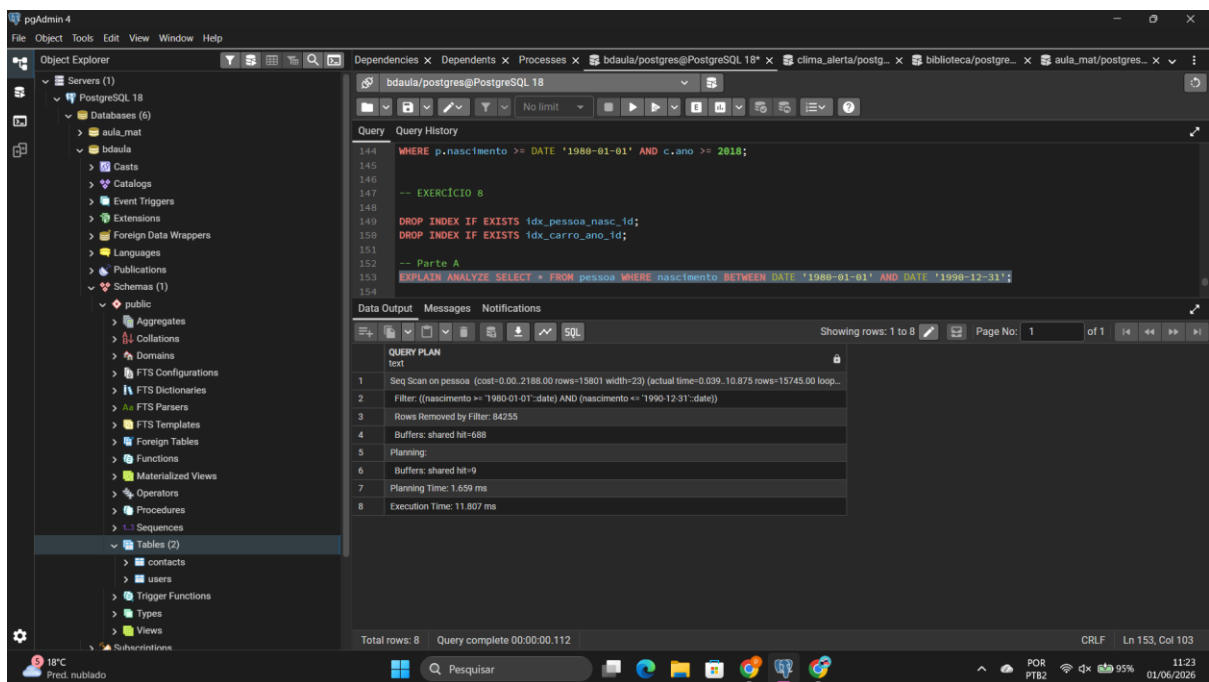


----- EXERCÍCIO 6 -----

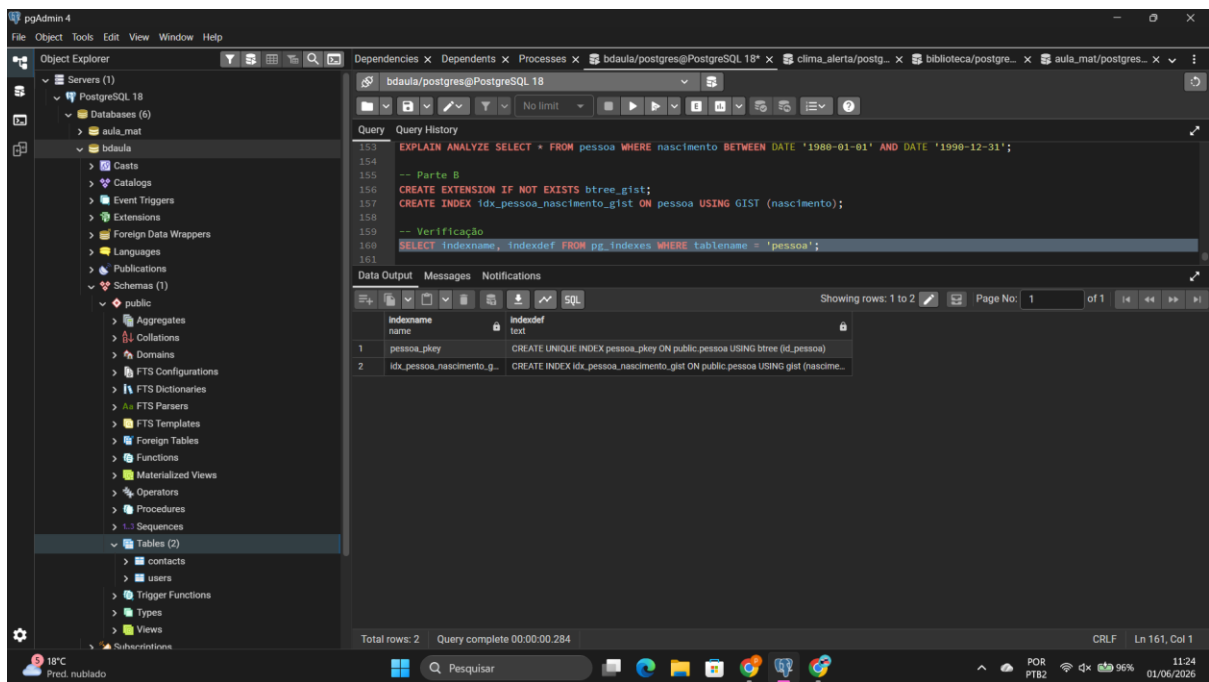


---- EXERCICIO 8 ----

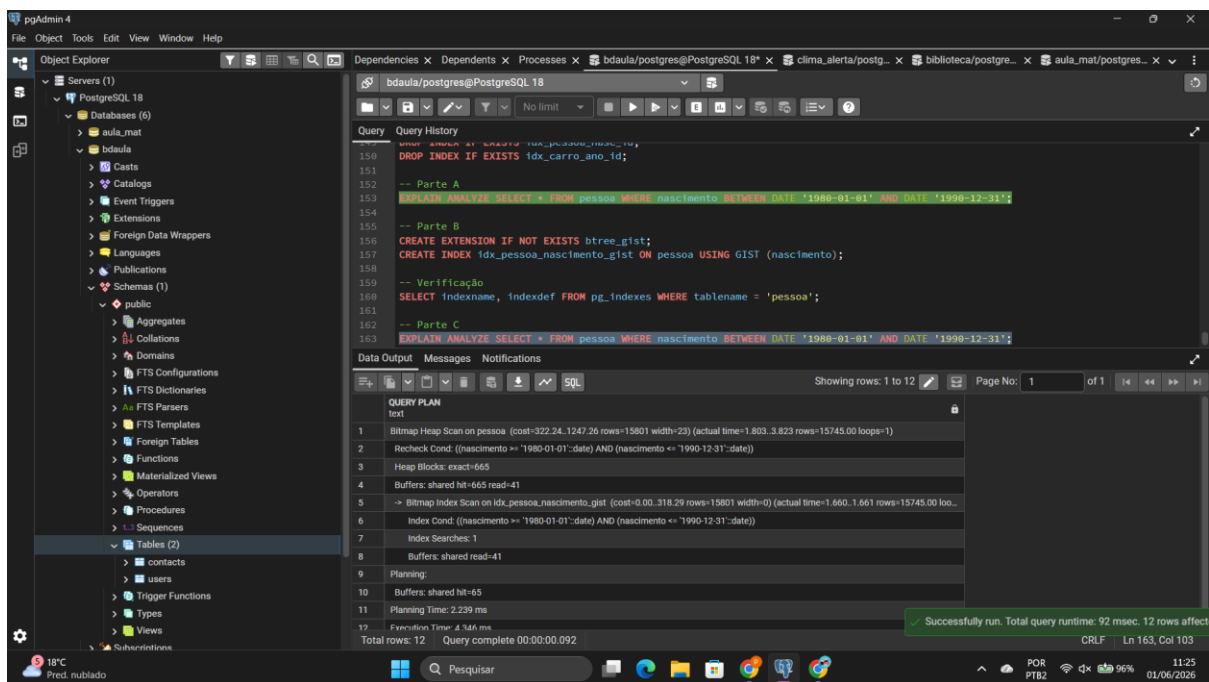
PARTE A



*PARTE B *



* PARTE C *



**COMANDOS SQL **

-- EXERCÍCIO 8

DROP INDEX IF EXISTS idx_pessoa_nasc_id; DROP INDEX IF EXISTS idx_carro_ano_id;

-- Parte A EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nascimento BETWEEN DATE '1988-01-01' AND DATE '1990-12-31';

-- Parte B CREATE EXTENSION IF NOT EXISTS btree_gist; CREATE INDEX idx_pessoa_nascimento_gist ON pessoa USING GIST (nascimento);

```
-- Verificação SELECT indexname, indexdef FROM pg_indexes WHERE tablename =  
'pessoa';
```

```
-- Parte C EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nascimento BETWEEN  
DATE '1980-01-01' AND DATE '1990-12-31';
```

--- ANÁLISE ---

1. **Qual plano foi utilizado antes da criação do índice GIST?** o índice B-tree se você esqueceu de apagá-lo
2. **O índice GIST foi utilizado após a criação?** Sim, o plano mudará para Index Scan ou Bitmap Index Scan usando `idx_pessoa_nascimento_gist`.
3. **Houve melhora no tempo de execução?** Sim, em relação ao Seq Scan.
Porém, vale a observação teórica: para dados escalares simples como DATE, o B-tree nativo costuma ser ainda mais eficiente e leve que o GIST, que é mais focado em dados geométricos e complexos.